



Data Bugs

A large percentage of the bugs found in software are due to improper data handling. This column analyses a few important types of 'data bugs' as well as some well-known historical examples.

Every program manipulates data. Mistakes or problems in this data manipulation can result in numerous bugs. Data bugs can occur due to a number of reasons. For instance, because of the wrong choice of data types (e.g., using an integer instead of float, and vice versa) or the loss of data due to conversions between data types (e.g., downcast from a float value to an integer value). They could also occur because of implementation constraints in handling certain data type values (e.g., integers will overflow if the value is beyond the range it can hold), and the peculiarities of language constructs in handling certain data types (e.g., bugs because of boxing and unboxing operations in languages like Java). We'll understand some of these problems with the help of coding or historic examples.

Perhaps Y2K is the most widely known example of data-related bugs. Date- and time-related problems seem to be common in computer systems. For example, consider the problem of the year 2038 in UNIX systems, which is the result of having chosen a shorter data type than required. Most UNIX/Linux systems use a 32-bit signed integer to represent time (*typedef*ed as *time_t* in C). For each second, the integer is incremented. As we know, the range of values a signed integer can represent is -2^{31} to $+2^{31} - 1$, which is -2147483648 to 2147483647 . This represents the time starting from 00:00:00 January 1, 1970 (the time when UNIX was developed) to 3:14:07 January 19, 2038. So, once the time counter value reaches 2147483647, it will overflow to -2147483648 .

This is known as 'The Year 2038 Problem'. For example, it can cause bugs and problems in the applications that use system time for calculations. Assume that you apply for a 30-year home loan, and the application your bank uses runs on a UNIX/Linux machine and makes use of this system time. The calculation of your loan's interest will give wrong results because of the Year 2038 problem. If the *long*

type were used for storing the time, we could have avoided this problem—the range of the long data type is perhaps sufficient for representing the time for thousands of years into the future. Hence, this is classified as a data bug.

The peculiarities of data types, and their behaviour in computers, can also cause bugs. A good example is the behaviour of floating-point numbers, which could often be unintuitive for programmers. 'Catastrophic cancellation' occurs when we subtract two nearly equal floating-point numbers. Consider the values $x = 0.54617$ and $y = 0.54601$; the exact difference in value is $d = 0.00016$. However, if four-digit arithmetic with rounding is used, then, $x' = 0.5462$, $y' = 0.5460$ and $d' = 0.0002$. Hence the relative error is:

$$|d - d'| / |d| = 0.25$$

This is quite large! When a large number of computations are performed with such an error, the final accumulated error can be huge. To give a historical example, in 1984, it was discovered that the Vancouver stock exchange was under-valued by 48 per cent, compared to its real value, because of an accumulating error in using floating-point numbers! This problem was because of the slow accumulation of round-off errors over time. The computation was performed using three decimal places instead of four; and instead of rounding, truncation was used.

Because digital computers have to store numbers in a fixed-size memory area (say 4 bytes for an integer), there are a range of values they cannot store; when you attempt to store values outside the range, overflow can occur. Another difficulty in storing values is that the range of values an integer can store is not symmetrical. For example, a signed byte (it has 8 bits of storage space) can store the range of values -128 to $+127$. Note the asymmetry in these two values—it is -128 and